SQL | MYSQL | MARKETPLACE ANALYTICS

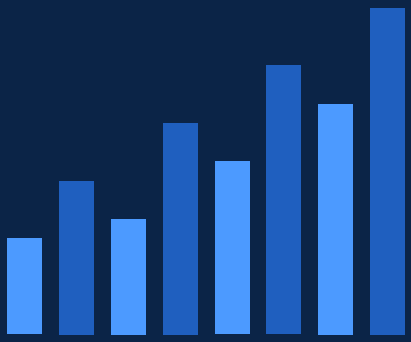
Marketplace Transactions

Commercial Performance, Customers & Cohort Quality

case_study_1.marketplace_transactions | 900 rows | 235 customers | Jan - Dec 2025

Data Analytics Portfolio Project

5 analysis sections | CTEs | Window functions | Cohort analysis



Project Overview

This case study uses MySQL to analyse 900 marketplace transactions from 2025. It rebuilds the monthly commercial bridge (GMV, delivered GMV, discounts, refunds, fees and a contribution proxy), measures customer concentration and repeat purchasing, tests discount dependency by category and channel, builds acquisition-month cohorts with 90-day economics, and quantifies the break-even volume decline for a 10% reduction in discount expense.

900	235	561,318.00	447,907.10	390,592.92
TRANSACTIONS	CUSTOMERS	TOTAL GMV	DELIVERED GMV	CONTRIBUTION PROXY

Totals are calculated from the source CSV across all 900 transaction lines (all order statuses). Contribution proxy counts delivered orders only.

Dataset at a Glance

order_status	lines
Delivered	723
Returned	76
Cancelled	64
Pending	37

product_category	lines
Beauty	194
Home & Kitchen	187
Electronics	186
Fashion	170
Sports	163

acquisition_channel	lines
Organic	199
Referral	193
Social	175
Affiliate	169
Paid Search	164

customer_segment	lines
Standard	576
Premium	238
High Value	86

Dataset Schema

Column	Type	Description
`order_id`	String	Order identifier
`customer_id`	String	Customer identifier
`seller_id`	String	Seller identifier
`product_category`	String	Beauty, Electronics, Fashion, Home & Kitchen, Sports
`order_timestamp`	Date	When the order was placed
`customer_acquisition_date`	Date	When the customer was acquired
`acquisition_channel`	String	Affiliate, Organic, Paid Search, Referral, Social
`order_status`	String	Delivered, Returned, Cancelled or Pending
`quantity`	Integer	Units ordered
`unit_list_price`	Float	List price per unit
`unit_selling_price`	Float	Selling price per unit
`discount_amount`	Float	Discount applied
`refund_amount`	Float	Amount refunded
`payment_fee`	Float	Payment processing fee
`fulfilment_cost`	Float	Fulfilment cost
`seller_commission_rate`	Float	Commission rate on sales
`customer_segment`	String	High Value, Premium or Standard

The MySQL table shown in Step 1 has these 17 columns; the source CSV also carries a transaction_id column.

Metric Definitions

GMV	quantity x unit_selling_price - discount_amount
Delivered GMV	GMV restricted to order_status = Delivered
Cancelled / Returned GMV	GMV restricted to Cancelled or Returned orders
Seller Commission Proxy	Delivered GMV x seller_commission_rate
Contribution Proxy	Delivered GMV - refunds - payment fees - fulfilment costs
Concentration band	Customers ranked by delivered GMV with NTILE(100); Top 1%, 5% and 10% of customers
Discount rate	Total discount / total list value (quantity x unit_list_price)
90-day contribution	Contribution from delivered orders within 90 days of the first delivered purchase
Break-even decline	100 x discount saving from a 10% cut / current contribution

Key Findings

Contribution moves with delivered GMV. Delivered GMV peaked in July (47,015.45) and bottomed in October (24,875.36); Step 17 returned no month with positive GMV growth and negative contribution growth.

June was the weakest month for returns. Returned GMV (10,124.94) and refunds (8,236.48) were the highest of the year, and delivered GMV fell 32.83%.

The biggest customers are slightly more profitable. The top 10% of customers generate 34.10% of delivered GMV and 35.62% of contribution; no customer above the 90th percentile has a negative contribution.

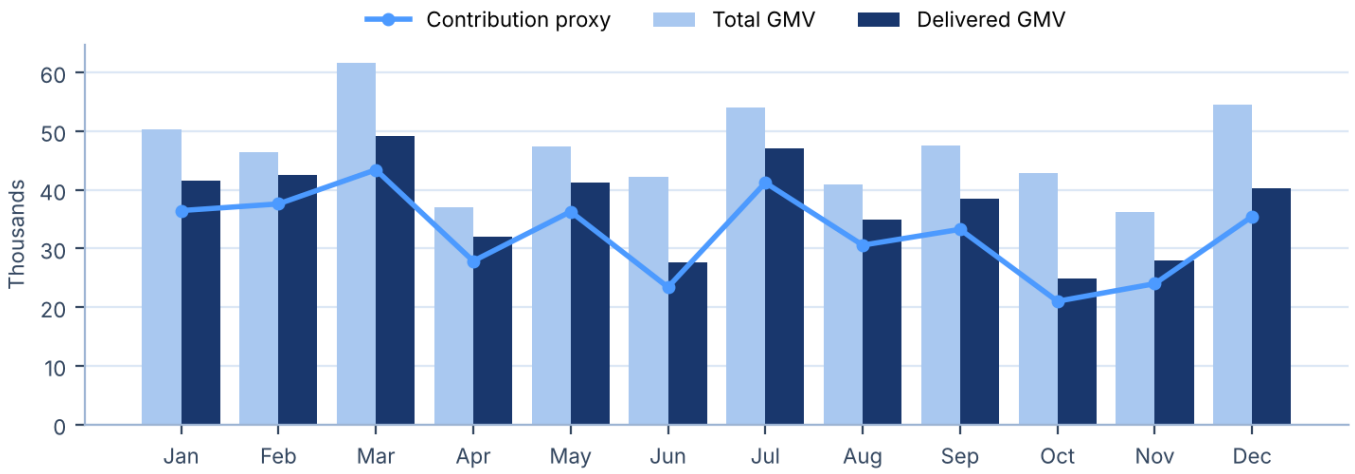
Repeat purchasing is strong. 191 of 226 delivered-order customers (84.51%) bought at least twice. Referral has the highest repeat rate (39.22%), Affiliate the lowest (18.69%).

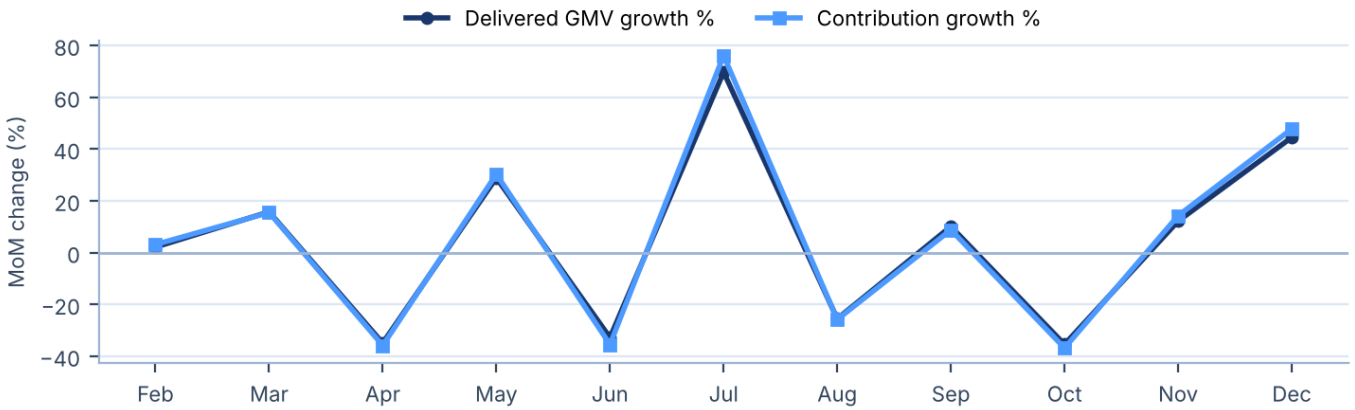
Discounting is highest in Beauty and Affiliate. Beauty carries a 16.53% discount rate and the Affiliate channel 17.06%; Sports (13.22%) and Social (14.04%) are lowest. Every category is rated Low commercial risk.

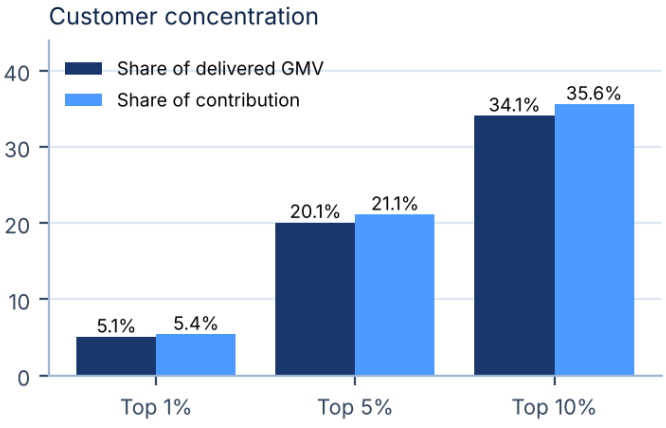
Sports has the least tolerance for volume decline. Its break-even decline is 2.09% versus 3.01% for Beauty, so a 10% discount cut is erased fastest there.

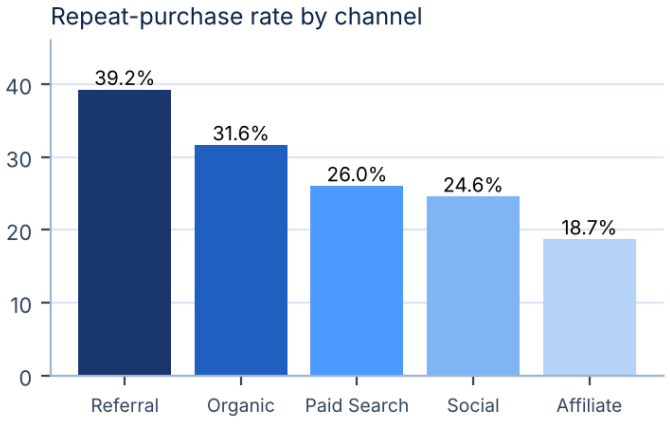
Cohort data is mostly complete. 119 acquisition-month and channel cohorts were built; 7 (October and November 2025) have incomplete 90-day windows. Only 1 complete cohort (Organic, 2023-04) shows 90-day contribution below first-purchase contribution.

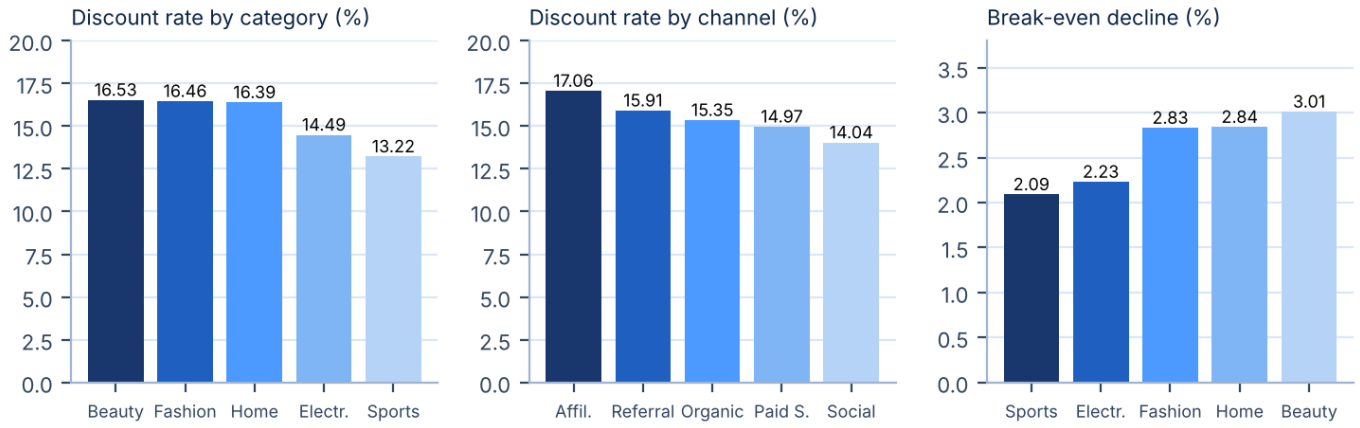
Visual Summary











Charts are drawn from the Section 1 (Steps 15, 16, 19), Section 2 (Step 2), Section 3 (Steps 1, 2), Section 4 (Step 5) and Section 5 (Step 4) outputs.

SQL Techniques Demonstrated

DATE_FORMAT month bucketing	Conditional aggregation with CASE WHEN
CTEs for multi-stage logic	LAG() for month-over-month growth
NTILE and CUME_DIST percentile bands	ROW_NUMBER for first-purchase detection
Cohort and 90-day window analysis	Scenario and break-even modelling

0 Database Setup

SQL QUERY

```sql
CREATE DATABASE case_study_1;

USE case_study_1;

SELECT * FROM marketplace_transactions;

SHOW TABLES ;
```

SECTION 1

Reconstruct the commercial performance bridge

Business question. Calculate monthly GMV, delivered GMV, cancelled GMV, returned GMV, discounts, refunds, seller commission, payment fees, fulfilment costs, and contribution proxy. Compare month-over-month growth in order count, GMV, delivered GMV, and contribution proxy. Identify months with positive GMV growth but negative contribution growth.

1.1 Check data

SQL QUERY

```
SELECT *
FROM marketplace_transactions
LIMIT 10;
```

OUTPUT (first 10 rows, two parts)

Part A: identifiers, dates and status

order_id	customer_id	seller_id	product_ category	order_ timestamp	customer_ acquisition_ date	acquisition_ channel	order_status
O00411	C0094	S029	Sports	14-08-2025	06-07-2025	Affiliate	Delivered
O00614	C0006	S027	Home & Kitchen	12-02-2025	19-05-2025	Social	Delivered
O00040	C0097	S030	Sports	30-09-2025	29-01-2023	Paid Search	Returned
O00062	C0203	S002	Electronics	20-09-2025	25-08-2025	Referral	Delivered
O00163	C0048	S023	Electronics	07-04-2025	09-04-2024	Social	Delivered
O00640	C0234	S049	Home & Kitchen	20-12-2025	29-05-2023	Organic	Cancelled
O00203	C0086	S069	Beauty	17-12-2025	23-06-2024	Social	Delivered
O00060	C0073	S023	Beauty	10-03-2025	24-04-2024	Social	Cancelled
O00256	C0167	S005	Sports	28-03-2025	24-03-2024	Affiliate	Delivered
O00385	C0104	S023	Beauty	19-04-2025	30-10-2024	Social	Delivered

Part B: quantities, prices, costs and segment

order_id	quantity	unit_list_price	unit_selling_price	discount_amount	refund_amount	payment_fee	fulfilment_cost	seller_commission_rate	customer_segment
O00411	2	412.77	393.72	38.09	0.00	21.24	46.04	0.122	High Value
O00614	1	437.00	367.67	69.33	0.00	6.92	67.84	0.153	Premium
O00040	1	321.99	308.42	13.57	230.58	6.46	101.56	0.079	High Value
O00062	1	135.62	119.14	16.48	0.00	2.88	36.17	0.122	Premium
O00163	1	1,064.24	945.68	118.56	0.00	20.99	96.76	0.146	Standard
O00640	2	1,510.14	1,176.15	667.98	0.00	42.13	66.82	0.149	Premium
O00203	1	469.51	442.76	26.75	0.00	14.10	71.13	0.107	Standard
O00060	1	141.29	131.44	9.85	0.00	3.12	85.58	0.081	Standard
O00256	1	895.32	736.66	158.66	0.00	17.20	99.32	0.157	Standard
O00385	1	393.45	346.20	47.25	0.00	11.61	15.75	0.145	Standard

1.2 Monthly GMV Analysis

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(quantity * unit_selling_price) AS gross_sales,
    SUM(discount_amount) AS total_discount,
    SUM(quantity * unit_selling_price - discount_amount) AS total_gmv
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month;
```

OUTPUT

month	gross_sales	total_discount	total_gmv
2025-01	60,366.40	10,025.80	50,340.60
2025-02	57,059.55	10,602.83	46,456.72
2025-03	76,117.20	14,404.60	61,712.60
2025-04	46,238.29	9,225.72	37,012.57
2025-05	56,198.01	8,730.23	47,467.78
2025-06	51,217.53	9,040.41	42,177.12
2025-07	65,040.74	11,002.45	54,038.29
2025-08	49,339.82	8,355.08	40,984.74
2025-09	57,912.42	10,341.69	47,570.73
2025-10	51,812.34	8,995.98	42,816.36
2025-11	46,851.01	10,637.18	36,213.83
2025-12	69,120.01	14,593.35	54,526.66

1.3 Monthly Delivered GMV

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(
        CASE
            WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0
        END
    ) AS delivered_gmv
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month;
```

OUTPUT

month	delivered_gmv
2025-01	41,622.30
2025-02	42,502.85
2025-03	49,186.15
2025-04	32,005.37
2025-05	41,244.88
2025-06	27,704.86
2025-07	47,015.45
2025-08	34,999.66
2025-09	38,529.06
2025-10	24,875.36
2025-11	27,903.66
2025-12	40,317.50

1.4 Cancelled GMV

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(
        CASE
            WHEN order_status = 'Cancelled'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0
        END
    ) AS cancelled_gmv
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month;
```

OUTPUT

month	cancelled_gmv
2025-01	5,288.67
2025-02	676.13
2025-03	6,686.20
2025-04	1,363.19
2025-05	1,080.45
2025-06	3,939.08
2025-07	2,029.46
2025-08	3,014.34
2025-09	4,267.44
2025-10	4,499.51
2025-11	1,314.99
2025-12	3,446.44

1.5 Returned GMV

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(
        CASE
            WHEN order_status = 'Returned'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0
        END
    ) AS returned_gmv
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month;
```

OUTPUT

month	returned_gmv
2025-01	1,457.70
2025-02	3,211.55
2025-03	4,438.13
2025-04	1,159.53
2025-05	4,051.34
2025-06	10,124.94
2025-07	3,345.50
2025-08	2,316.36
2025-09	3,732.00
2025-10	5,778.27
2025-11	5,671.41
2025-12	4,334.84

1.6 Discounts

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(discount_amount) AS total_discount
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month;
```

OUTPUT

month	total_discount
2025-01	10,025.80
2025-02	10,602.83
2025-03	14,404.60
2025-04	9,225.72
2025-05	8,730.23
2025-06	9,040.41
2025-07	11,002.45
2025-08	8,355.08
2025-09	10,341.69
2025-10	8,995.98
2025-11	10,637.18
2025-12	14,593.35

1.7 Refunds

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(refund_amount) AS total_refunds
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month;
```

OUTPUT

month	total_refunds
2025-01	1,412.26
2025-02	2,865.51
2025-03	2,882.47
2025-04	1,226.17
2025-05	2,709.53
2025-06	8,236.48
2025-07	2,623.37
2025-08	1,260.60
2025-09	3,390.61
2025-10	4,780.92
2025-11	3,868.39
2025-12	2,688.98

1.8 Seller Commission

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(
        CASE
            WHEN order_status = 'Delivered'
            THEN (quantity * unit_selling_price - discount_amount)
                 * seller_commission_rate
            ELSE 0
        END
    ) AS seller_commission_proxy
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month;
```

OUTPUT

month	seller_commission_proxy
2025-01	5,678.34
2025-02	5,556.19
2025-03	6,608.44
2025-04	4,020.64
2025-05	5,184.40
2025-06	3,185.15
2025-07	5,120.16
2025-08	4,192.68
2025-09	4,844.16
2025-10	3,195.33
2025-11	3,164.16
2025-12	5,002.20

1.9 Payment Fees

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(payment_fee) AS total_payment_fees
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month ASC;
```

OUTPUT

month	total_payment_fees
2025-01	1,479.15
2025-02	1,387.32
2025-03	1,719.82
2025-04	1,018.41
2025-05	1,395.51
2025-06	1,250.83
2025-07	1,609.10
2025-08	1,176.28
2025-09	1,405.50
2025-10	1,201.75
2025-11	1,132.59
2025-12	1,721.44

1.10 Fulfilment costs

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(fulfilment_cost) AS total_fulfilment_cost
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month ASC;
```

OUTPUT

month	total_ fulfilment_ cost
2025-01	4,876.27
2025-02	4,256.52
2025-03	5,437.35
2025-04	4,009.22
2025-05	4,651.18
2025-06	4,797.74
2025-07	5,152.56
2025-08	4,021.82
2025-09	5,035.83
2025-10	4,130.13
2025-11	3,720.36
2025-12	4,600.82

1.11 Contribution Proxy

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    SUM(
        CASE
            WHEN order_status = 'Delivered'
            THEN (quantity * unit_selling_price - discount_amount)
                 - refund_amount
                 - payment_fee
                 - fulfilment_cost
            ELSE 0
        END
    ) AS contribution_proxy
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month ASC;
```

OUTPUT

month	contribution_ proxy
2025-01	36,467.04
2025-02	37,603.80
2025-03	43,421.82
2025-04	27,856.92
2025-05	36,287.68
2025-06	23,412.00
2025-07	41,221.17
2025-08	30,577.97
2025-09	33,264.34
2025-10	21,010.57
2025-11	24,003.67
2025-12	35,465.94

1.12 Monthly Order Count

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
    COUNT(DISTINCT order_id) AS total_orders
FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month ASC;
```

OUTPUT

month	total_orders
2025-01	74
2025-02	71
2025-03	81
2025-04	61
2025-05	71
2025-06	69
2025-07	78
2025-08	65
2025-09	81
2025-10	68
2025-11	65
2025-12	77

1.13 Month-over-Month Order Count Growth

SQL QUERY

```sql
WITH monthly_orders AS (
    SELECT
        DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
        COUNT(DISTINCT order_id) AS total_orders
    FROM marketplace_transactions
    GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
),
order_growth AS (
    SELECT
        month,
        total_orders,
        LAG(total_orders) OVER (ORDER BY month) AS previous_month_orders
    FROM monthly_orders
)
SELECT
    month,
    total_orders,
    previous_month_orders,
    total_orders - previous_month_orders AS order_count_change,
    ROUND(
        100.0 * (total_orders - previous_month_orders)
        / NULLIF(previous_month_orders, 0),
        2
    ) AS order_growth_pct
FROM order_growth
ORDER BY month ASC;
```

OUTPUT

month	total_orders	previous_month_orders	order_count_change	order_growth_pct
2025-01	74	*NULL*	*NULL*	*NULL*
2025-02	71	74	-3	-4.05
2025-03	81	71	10	14.08
2025-04	61	81	-20	-24.69
2025-05	71	61	10	16.39
2025-06	69	71	-2	-2.82
2025-07	78	69	9	13.04
2025-08	65	78	-13	-16.67
2025-09	81	65	16	24.62
2025-10	68	81	-13	-16.05
2025-11	65	68	-3	-4.41
2025-12	77	65	12	18.46

NULL = no previous month (January is the first month in the data).

1.14 Month-over-Month GMV Growth

SQL QUERY

```sql
WITH monthly_gmv AS (
    SELECT
        DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
        SUM(quantity * unit_selling_price - discount_amount) AS total_gmv
    FROM marketplace_transactions
    GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
),
gmv_growth AS (
    SELECT
        month,
        total_gmv,
        LAG(total_gmv) OVER (ORDER BY month) AS previous_month_gmv
    FROM monthly_gmv
)
SELECT
    month,
    ROUND(total_gmv, 2) AS total_gmv,
    ROUND(previous_month_gmv, 2) AS previous_month_gmv,
    ROUND(total_gmv - previous_month_gmv, 2) AS gmv_change,
    ROUND(
        100.0 * (total_gmv - previous_month_gmv)
        / NULLIF(previous_month_gmv, 0),
        2
    ) AS gmv_growth_pct
FROM gmv_growth
ORDER BY month ASC;
```

OUTPUT

month	total_gmv	previous_month_gmv	gmv_change	gmv_growth_pct
2025-01	50,340.60	*NULL*	*NULL*	*NULL*
2025-02	46,456.72	50,340.60	-3,883.88	-7.72
2025-03	61,712.60	46,456.72	15,255.88	32.84
2025-04	37,012.57	61,712.60	-24,700.03	-40.02
2025-05	47,467.78	37,012.57	10,455.21	28.25
2025-06	42,177.12	47,467.78	-5,290.66	-11.15
2025-07	54,038.29	42,177.12	11,861.17	28.12
2025-08	40,984.74	54,038.29	-13,053.55	-24.16
2025-09	47,570.73	40,984.74	6,585.99	16.07
2025-10	42,816.36	47,570.73	-4,754.37	-9.99
2025-11	36,213.83	42,816.36	-6,602.53	-15.42
2025-12	54,526.66	36,213.83	18,312.83	50.57

1.15 Month-over-Month Delivered GMV Growth

SQL QUERY

```sql
WITH monthly_delivered AS (
    SELECT
        DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN quantity * unit_selling_price - discount_amount
                ELSE 0
            END
        ) AS delivered_gmv
    FROM marketplace_transactions
    GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
),
delivered_growth AS (
    SELECT
        month,
        delivered_gmv,
        LAG(delivered_gmv) OVER (ORDER BY month) AS previous_month_delivered_gmv
    FROM monthly_delivered
)
SELECT
    month,
    ROUND(delivered_gmv, 2) AS delivered_gmv,
    ROUND(previous_month_delivered_gmv, 2) AS previous_month_delivered_gmv,
    ROUND(delivered_gmv - previous_month_delivered_gmv, 2) AS gmv_change,
    ROUND(
        100.0 * (delivered_gmv - previous_month_delivered_gmv)
        / NULLIF(previous_month_delivered_gmv, 0),
        2
    ) AS delivered_gmv_growth_pct
FROM delivered_growth
ORDER BY month ASC;
```

OUTPUT

month	delivered_gmv	previous_month_delivered_gmv	gmv_change	delivered_gmv_growth_pct
2025-01	41,622.30	*NULL*	*NULL*	*NULL*
2025-02	42,502.85	41,622.30	880.55	2.12
2025-03	49,186.15	42,502.85	6,683.30	15.72
2025-04	32,005.37	49,186.15	-17,180.78	-34.93
2025-05	41,244.88	32,005.37	9,239.51	28.87
2025-06	27,704.86	41,244.88	-13,540.02	-32.83
2025-07	47,015.45	27,704.86	19,310.59	69.70
2025-08	34,999.66	47,015.45	-12,015.79	-25.56
2025-09	38,529.06	34,999.66	3,529.40	10.08
2025-10	24,875.36	38,529.06	-13,653.70	-35.44
2025-11	27,903.66	24,875.36	3,028.30	12.17
2025-12	40,317.50	27,903.66	12,413.84	44.49

1.16 Month-over-Month Contribution Proxy Growth

SQL QUERY

```sql
WITH monthly_contribution AS (
    SELECT
        DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN (quantity * unit_selling_price - discount_amount)
                     - refund_amount
                     - payment_fee
                     - fulfilment_cost
                ELSE 0
            END
        ) AS contribution_proxy
    FROM marketplace_transactions
    GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
),
contribution_growth AS (
    SELECT
        month,
        contribution_proxy,
        LAG(contribution_proxy) OVER (ORDER BY month) AS previous_month_contribution
    FROM monthly_contribution
)
SELECT
    month,
    ROUND(contribution_proxy, 2) AS contribution_proxy,
    ROUND(previous_month_contribution, 2) AS previous_month_contribution,
    ROUND(contribution_proxy - previous_month_contribution, 2) AS contribution_change,
    ROUND(
        100.0 * (contribution_proxy - previous_month_contribution)
        / NULLIF(previous_month_contribution, 0),
        2
    ) AS contribution_growth_pct
FROM contribution_growth
ORDER BY month ASC;
```

OUTPUT

month	contribution_ proxy	previous_month_ contribution	contribution_ change	contribution_ growth_pct
2025-01	36,467.04	*NULL*	*NULL*	*NULL*
2025-02	37,603.80	36,467.04	1,136.76	3.12
2025-03	43,421.82	37,603.80	5,818.02	15.47
2025-04	27,856.92	43,421.82	-15,564.90	-35.85
2025-05	36,287.68	27,856.92	8,430.76	30.26
2025-06	23,412.00	36,287.68	-12,875.68	-35.48
2025-07	41,221.17	23,412.00	17,809.17	76.07
2025-08	30,577.97	41,221.17	-10,643.20	-25.82
2025-09	33,264.34	30,577.97	2,686.37	8.79
2025-10	21,010.57	33,264.34	-12,253.77	-36.84
2025-11	24,003.67	21,010.57	2,993.10	14.25
2025-12	35,465.94	24,003.67	11,462.27	47.75

1.17 Identify Months with Positive GMV Growth but Negative Contribution Growth

SQL QUERY

```sql
WITH monthly_metrics AS (
    SELECT
        DATE_FORMAT(order_timestamp, '%Y-%m') AS month,

        SUM(quantity * unit_selling_price - discount_amount) AS total_gmv,

        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN (quantity * unit_selling_price - discount_amount)
                     - refund_amount
                     - payment_fee
                     - fulfilment_cost
                ELSE 0
            END
        ) AS contribution_proxy

    FROM marketplace_transactions
    GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
),
growth_metrics AS (
    SELECT
        month,
        total_gmv,
        contribution_proxy,

        100.0 * (
            total_gmv - LAG(total_gmv) OVER (ORDER BY month)
        ) / NULLIF(
            LAG(total_gmv) OVER (ORDER BY month), 0
        ) AS gmv_growth_pct,

        100.0 * (
            contribution_proxy -
            LAG(contribution_proxy) OVER (ORDER BY month)
        ) / NULLIF(
            LAG(contribution_proxy) OVER (ORDER BY month), 0
        ) AS contribution_growth_pct

    FROM monthly_metrics
)
SELECT
    month,
    ROUND(gmv_growth_pct, 2) AS gmv_growth_pct,
    ROUND(contribution_growth_pct, 2) AS contribution_growth_pct
FROM growth_metrics
WHERE gmv_growth_pct > 0
  AND contribution_growth_pct < 0
ORDER BY month;
```

OUTPUT

month	gmv_growth_pct	contribution_ growth_pct

Empty set: no month had positive GMV growth together with negative contribution growth.

1.18 Check Both Growth Metrics

SQL QUERY

```sql
WITH monthly_metrics AS (
    SELECT
        DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
        SUM(quantity * unit_selling_price - discount_amount) AS total_gmv,
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN (quantity * unit_selling_price - discount_amount)
                     - refund_amount
                     - payment_fee
                     - fulfilment_cost
                ELSE 0
            END
        ) AS contribution_proxy
    FROM marketplace_transactions
    GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
),
growth_metrics AS (
    SELECT
        month,
        total_gmv,
        contribution_proxy,
        LAG(total_gmv) OVER (ORDER BY month) AS previous_gmv,
        LAG(contribution_proxy) OVER (ORDER BY month) AS previous_contribution
    FROM monthly_metrics
)
SELECT
    month,
    ROUND(100.0 * (total_gmv - previous_gmv)
        / NULLIF(previous_gmv, 0), 2) AS gmv_growth_pct,
    ROUND(100.0 * (contribution_proxy - previous_contribution)
        / NULLIF(previous_contribution, 0), 2) AS contribution_growth_pct
FROM growth_metrics
ORDER BY month;
```

OUTPUT

month	gmv_growth_pct	contribution_growth_pct
2025-01	*NULL*	*NULL*
2025-02	-7.72	3.12
2025-03	32.84	15.47
2025-04	-40.02	-35.85
2025-05	28.25	30.26
2025-06	-11.15	-35.48
2025-07	28.12	76.07
2025-08	-24.16	-25.82
2025-09	16.07	8.79
2025-10	-9.99	-36.84
2025-11	-15.42	14.25
2025-12	50.57	47.75

1.19 Monthly GMV, Delivered GMV , Contribution Summary

SQL QUERY

```sql
SELECT
    DATE_FORMAT(order_timestamp, '%Y-%m') AS month,

    ROUND(
        SUM(quantity * unit_selling_price - discount_amount), 2
    ) AS total_gmv,

    ROUND(
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN quantity * unit_selling_price - discount_amount
                ELSE 0
            END
        ), 2
    ) AS delivered_gmv,

    ROUND(
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN (quantity * unit_selling_price - discount_amount)
                     - refund_amount
                     - payment_fee
                     - fulfilment_cost
                ELSE 0
            END
        ), 2
    ) AS contribution_proxy

FROM marketplace_transactions
GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
ORDER BY month ASC;
```

OUTPUT

month	total_gmv	delivered_gmv	contribution_ proxy
2025-01	50,340.60	41,622.30	36,467.04
2025-02	46,456.72	42,502.85	37,603.80
2025-03	61,712.60	49,186.15	43,421.82
2025-04	37,012.57	32,005.37	27,856.92
2025-05	47,467.78	41,244.88	36,287.68
2025-06	42,177.12	27,704.86	23,412.00
2025-07	54,038.29	47,015.45	41,221.17
2025-08	40,984.74	34,999.66	30,577.97
2025-09	47,570.73	38,529.06	33,264.34
2025-10	42,816.36	24,875.36	21,010.57
2025-11	36,213.83	27,903.66	24,003.67
2025-12	54,526.66	40,317.50	35,465.94

Final Final Query

SQL QUERY

```sql
USE case_study_1;

WITH monthly_metrics AS (
    SELECT
        DATE_FORMAT(order_timestamp, '%Y-%m') AS month,

        SUM(quantity * unit_selling_price - discount_amount)
            AS total_gmv,

        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0 END
        ) AS delivered_gmv,

        COUNT(DISTINCT
            CASE WHEN order_status = 'Delivered'
            THEN order_id END
        ) AS delivered_orders,

        SUM(discount_amount) AS discounts,

        SUM(refund_amount) AS refunds,

        SUM(
            CASE WHEN order_status = 'Cancelled'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0 END
        ) AS cancelled_gmv,

        SUM(
            CASE WHEN order_status = 'Returned'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0 END
        ) AS returned_gmv,

        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN (quantity * unit_selling_price - discount_amount)
                 * seller_commission_rate
            ELSE 0 END
        ) AS seller_commission_proxy,

        SUM(payment_fee) AS payment_fees,

        SUM(fulfilment_cost) AS fulfilment_costs,

        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN (quantity * unit_selling_price - discount_amount)
                 - refund_amount - payment_fee - fulfilment_cost
            ELSE 0 END
        ) AS contribution_proxy

    FROM marketplace_transactions
    GROUP BY DATE_FORMAT(order_timestamp, '%Y-%m')
),

growth_metrics AS (
    SELECT
        *,
        100.0 * (
            delivered_gmv - LAG(delivered_gmv) OVER (ORDER BY month)
        ) / NULLIF(
            LAG(delivered_gmv) OVER (ORDER BY month), 0
        ) AS gmv_growth_pct,

        100.0 * (
```

```
            contribution_proxy
            - LAG(contribution_proxy) OVER (ORDER BY month)
        ) / NULLIF(
            LAG(contribution_proxy) OVER (ORDER BY month), 0
        ) AS contribution_growth_pct

    FROM monthly_metrics
)

SELECT
    month,
    ROUND(delivered_gmv, 2) AS delivered_gmv,
    delivered_orders,
    ROUND(discounts, 2) AS discounts,
    ROUND(refunds, 2) AS refunds,
    ROUND(contribution_proxy, 2) AS contribution_proxy,
    ROUND(gmv_growth_pct, 2) AS gmv_growth_pct,
    ROUND(contribution_growth_pct, 2) AS contribution_growth_pct

FROM growth_metrics
ORDER BY month ASC;
```

OUTPUT

month	delivered_gmv	delivered_orders	discounts	refunds	contribution_proxy	gmv_growth_pct	contribution_growth_pct
2025-01	41,622.30	60	10,025.80	1,412.26	36,467.04	*NULL*	*NULL*
2025-02	42,502.85	61	10,602.83	2,865.51	37,603.80	2.12	3.12
2025-03	49,186.15	65	14,404.60	2,882.47	43,421.82	15.72	15.47
2025-04	32,005.37	53	9,225.72	1,226.17	27,856.92	-34.93	-35.85
2025-05	41,244.88	58	8,730.23	2,709.53	36,287.68	28.87	30.26
2025-06	27,704.86	52	9,040.41	8,236.48	23,412.00	-32.83	-35.48
2025-07	47,015.45	66	11,002.45	2,623.37	41,221.17	69.70	76.07
2025-08	34,999.66	55	8,355.08	1,260.60	30,577.97	-25.56	-25.82
2025-09	38,529.06	64	10,341.69	3,390.61	33,264.34	10.08	8.79
2025-10	24,875.36	49	8,995.98	4,780.92	21,010.57	-35.44	-36.84
2025-11	27,903.66	52	10,637.18	3,868.39	24,003.67	12.17	14.25
2025-12	40,317.50	59	14,593.35	2,688.98	35,465.94	44.49	47.75

In the final query, gmv_growth_pct is calculated on Delivered GMV (Step 15), while Steps 14, 17 and 18 use total GMV.

SECTION 2

Customer concentration and economic dependency

Business question. Calculate the share of delivered GMV and contribution proxy generated by the top 1%, 5%, and 10% of customers. Calculate repeat-purchase participation and identify customers whose delivered GMV is above the 90th percentile but whose contribution proxy is negative. Determine whether the largest customers are also the most profitable.

2.1 Calculate Customer-wise Delivered GMV and Contribution Proxy

SQL QUERY

```sql
SELECT
    customer_id,
    ROUND(
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN quantity * unit_selling_price - discount_amount
                ELSE 0
            END
        ), 2
    ) AS delivered_gmv,
    ROUND(
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN quantity * unit_selling_price
                     - discount_amount
                     - refund_amount
                     - payment_fee
                     - fulfilment_cost
                ELSE 0
            END
        ), 2
    ) AS contribution_proxy
FROM marketplace_transactions
GROUP BY customer_id
ORDER BY delivered_gmv DESC;
```

OUTPUT

customer_id	delivered_gmv	contribution_proxy
C0100	8,974.79	8,461.59
C0097	6,900.56	6,245.43
C0002	6,831.47	6,415.45
C0131	6,263.04	5,753.40
C0010	6,044.40	5,515.53
C0239	5,890.59	5,428.95
C0129	5,866.50	5,336.17
C0150	5,827.17	5,360.28
C0189	5,790.08	5,211.15
C0085	5,624.01	5,218.64
C0024	5,440.20	5,038.57
C0165	5,429.14	4,995.81
C0064	5,110.32	4,517.60
C0114	5,090.27	4,821.65
C0147	4,940.15	4,264.27

Preview: first 15 of 235 customers, ordered by delivered_gmv.

2.2 Customer Concentration

SQL QUERY

```sql
WITH customer_totals AS (
    SELECT
        customer_id,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0 END
        ) AS delivered_gmv,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price
                 - discount_amount
                 - refund_amount
                 - payment_fee
                 - fulfilment_cost
            ELSE 0 END
        ) AS contribution_proxy
    FROM marketplace_transactions
    GROUP BY customer_id
),
ranked AS (
    SELECT *,
        NTILE(100) OVER (
            ORDER BY delivered_gmv DESC
        ) AS pct_bucket
    FROM customer_totals
),
company AS (
    SELECT
        SUM(delivered_gmv) AS total_gmv,
        SUM(contribution_proxy) AS total_contribution
    FROM customer_totals
)
SELECT
    bands.concentration_band,
    COUNT(r.customer_id) AS customer_count,
    ROUND(SUM(r.delivered_gmv), 2) AS delivered_gmv,
    ROUND(SUM(r.contribution_proxy), 2) AS contribution_proxy,
    ROUND(
        100 * SUM(r.delivered_gmv) /
        NULLIF(c.total_gmv, 0), 2
    ) AS share_of_total_gmv_pct,
    ROUND(
        100 * SUM(r.contribution_proxy) /
        NULLIF(c.total_contribution, 0), 2
    ) AS share_of_total_contribution_pct
FROM (
    SELECT 1 AS bucket_limit, 'Top 1%' AS concentration_band
    UNION ALL SELECT 5, 'Top 5%'
    UNION ALL SELECT 10, 'Top 10%'
) AS bands
CROSS JOIN company c
LEFT JOIN ranked r
    ON r.pct_bucket <= bands.bucket_limit
GROUP BY
    bands.bucket_limit,
    bands.concentration_band,
    c.total_gmv,
    c.total_contribution
ORDER BY bands.bucket_limit;
```

OUTPUT

concentration_band	customer_count	delivered_gmv	contribution_proxy	share_of_total_gmv_pct	share_of_total_contribution_pct
Top 1%	3	22,706.82	21,122.47	5.07	5.41
Top 5%	15	90,022.69	82,584.49	20.10	21.14
Top 10%	30	152,736.03	139,119.76	34.10	35.62

2.3 Calculate Repeat Purchase Participation

SQL QUERY

```sql
WITH customer_orders AS (
    SELECT
        customer_id,
        COUNT(DISTINCT order_id) AS delivered_order_count
    FROM marketplace_transactions
    WHERE order_status = 'Delivered'
    GROUP BY customer_id
)
SELECT
    COUNT(*) AS total_customers,
    SUM(
        CASE
            WHEN delivered_order_count >= 2 THEN 1
            ELSE 0
        END
    ) AS repeat_customers,
    ROUND(
        100.0 * SUM(
            CASE
                WHEN delivered_order_count >= 2 THEN 1
                ELSE 0
            END
        ) / NULLIF(COUNT(*), 0),
        2
    ) AS repeat_purchase_participation_pct
FROM customer_orders;
```

OUTPUT

total_customers	repeat_customers	repeat_purchase_participation_pct
226	191	84.51

2.4 Identify High-Value Customers with Negative Contribution

SQL QUERY

```sql
WITH customer_totals AS (
    SELECT
        customer_id,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0 END
        ) AS delivered_gmv,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price
                 - discount_amount
                 - refund_amount
                 - payment_fee
                 - fulfilment_cost
            ELSE 0 END
        ) AS contribution_proxy,
        COUNT(DISTINCT CASE
            WHEN order_status = 'Delivered' THEN order_id
        END) AS delivered_order_count
    FROM marketplace_transactions
    GROUP BY customer_id
),
ranked_customers AS (
    SELECT *,
        CUME_DIST() OVER (
            ORDER BY delivered_gmv
        ) AS gmv_percentile
    FROM customer_totals
)
SELECT
    customer_id,
    ROUND(delivered_gmv, 2) AS delivered_gmv,
    ROUND(contribution_proxy, 2) AS contribution_proxy,
    delivered_order_count,
    ROUND(gmv_percentile, 2) AS gmv_percentile,
    'Yes' AS negative_contribution
FROM ranked_customers
WHERE gmv_percentile > 0.90
  AND contribution_proxy < 0
ORDER BY delivered_gmv DESC;
```

OUTPUT

customer_id	delivered_gmv	contribution_ proxy	delivered_ order_count	gmv_percentile	negative_ contribution

Empty set: no customers above the 90th percentile of delivered GMV have a negative contribution proxy.

SECTION 3

Discount dependency and commercial quality

Business question. Calculate discount rates by product category and acquisition channel. Identify categories where higher discount rates coincide with lower contribution per delivered order. Compare customer segments by average order value, repeat-purchase frequency, discount dependency, refund burden, and contribution proxy. Identify high-value transactions with weak post-discount economics.

3.1 Calculate Discount Rate by Product Category

SQL QUERY

```sql
SELECT
    product_category,
    ROUND(
        100.0 * SUM(discount_amount) /
        NULLIF(SUM(quantity * unit_list_price), 0),
        2
    ) AS discount_rate_pct
FROM marketplace_transactions
GROUP BY product_category
ORDER BY discount_rate_pct DESC;
```

OUTPUT

product_ category	discount_rate_ pct
Beauty	16.53
Fashion	16.46
Home & Kitchen	16.39
Electronics	14.49
Sports	13.22

3.2 Calculate Discount Rate by Acquisition Channel

SQL QUERY

```sql
SELECT
    acquisition_channel,
    ROUND(
        100.0 * SUM(discount_amount) /
        NULLIF(SUM(quantity * unit_list_price), 0),
        2
    ) AS discount_rate_pct
FROM marketplace_transactions
GROUP BY acquisition_channel
ORDER BY discount_rate_pct DESC;
```

OUTPUT

acquisition_channel	discount_rate_pct
Affiliate	17.06
Referral	15.91
Organic	15.35
Paid Search	14.97
Social	14.04

3.3 Analyse Category Commercial Quality

SQL QUERY

```sql
WITH category_metrics AS (
    SELECT
        product_category,
        SUM(discount_amount) AS total_discount,
        SUM(quantity * unit_list_price) AS total_list_value,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price
                 - discount_amount
                 - refund_amount
                 - payment_fee
                 - fulfilment_cost
            ELSE 0 END
        ) AS contribution_proxy,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN 1 ELSE 0 END
        ) AS delivered_orders,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN refund_amount ELSE 0 END
        ) AS delivered_refunds,
        SUM(
            CASE WHEN order_status = 'Delivered'
            THEN quantity * unit_selling_price - discount_amount
            ELSE 0 END
        ) AS delivered_gmv
    FROM marketplace_transactions
    GROUP BY product_category
)
SELECT
    product_category,
    ROUND(
        100.0 * total_discount /
        NULLIF(total_list_value, 0), 2
    ) AS discount_rate_pct,
    ROUND(
        contribution_proxy /
        NULLIF(delivered_orders, 0), 2
    ) AS contribution_per_order,
    ROUND(
        100.0 * delivered_refunds /
        NULLIF(delivered_gmv, 0), 2
    ) AS refund_rate_pct,
    CASE
        WHEN contribution_proxy / NULLIF(delivered_orders, 0) < 100
             OR 100.0 * delivered_refunds /
                NULLIF(delivered_gmv, 0) >= 8
            THEN 'High'
        WHEN contribution_proxy / NULLIF(delivered_orders, 0) < 200
             OR 100.0 * delivered_refunds /
                NULLIF(delivered_gmv, 0) >= 4
            THEN 'Medium'
        ELSE 'Low'
    END AS commercial_risk
FROM category_metrics
ORDER BY contribution_per_order ASC;
```

OUTPUT

product_category	discount_rate_pct	contribution_per_order	refund_rate_pct	commercial_risk
Fashion	16.46	447.51	0.00	Low
Sports	13.22	493.62	0.00	Low
Home & Kitchen	16.39	527.42	0.00	Low
Beauty	16.53	575.24	0.00	Low
Electronics	14.49	644.07	0.00	Low

3.4 Customer Segment Comparison — Average Order Value

SQL QUERY

```sql
SELECT
    customer_id,
    ROUND(
        AVG(quantity * unit_selling_price - discount_amount),
        2
    ) AS average_order_value
FROM marketplace_transactions
GROUP BY customer_id
ORDER BY average_order_value DESC;
```

OUTPUT

customer_id	average_order_value
C0130	4,543.86
C0088	2,623.35
C0091	2,360.09
C0100	1,906.73
C0085	1,874.67
C0110	1,785.30
C0002	1,707.87
C0114	1,696.76
C0237	1,695.23
C0134	1,494.19
C0045	1,414.80
C0010	1,322.50
C0142	1,311.88
C0072	1,306.04
C0024	1,301.73

Preview: first 15 of 235 customers.

3.5 Repeat-Purchase Frequency

SQL QUERY

```sql
SELECT
    customer_id,
    COUNT(*) AS delivered_order_count
FROM marketplace_transactions
WHERE order_status = 'Delivered'
GROUP BY customer_id
ORDER BY delivered_order_count DESC;
```

OUTPUT

customer_id	delivered_ order_count
C0147	8
C0151	8
C0170	8
C0189	8
C0064	7
C0087	7
C0097	7
C0108	7
C0127	7
C0129	7
C0157	7
C0171	7
C0238	7
C0005	6
C0006	6

Preview: first 15 of 226 customers.

3.6 Discount Dependency by Customer

SQL QUERY

```sql
SELECT
    customer_id,
    ROUND(
        100.0 * SUM(discount_amount) /
        NULLIF(SUM(quantity * unit_list_price), 0),
        2
    ) AS discount_dependency_pct
FROM marketplace_transactions
GROUP BY customer_id
ORDER BY discount_dependency_pct DESC;
```

OUTPUT

customer_id	discount_ dependency_pct
C0116	45.00
C0178	33.88
C0208	33.27
C0229	32.34
C0066	31.91
C0146	31.58
C0017	31.29
C0123	31.28
C0099	31.13
C0025	30.96
C0206	30.58
C0147	29.54
C0207	29.00
C0035	28.54
C0104	28.35

Preview: first 15 of 235 customers.

3.7 Refund Burden by Customer

SQL QUERY

```sql
SELECT
    customer_id,
    ROUND(SUM(refund_amount), 2) AS refund_burden
FROM marketplace_transactions
GROUP BY customer_id
ORDER BY refund_burden DESC;
```

OUTPUT

customer_id	refund_burden
C0088	3,882.61
C0123	2,748.96
C0152	1,811.03
C0199	1,709.89
C0075	1,436.71
C0009	1,265.76
C0046	1,211.81
C0108	1,190.98
C0166	1,087.70
C0183	1,072.49
C0202	1,052.27
C0140	997.50
C0016	989.09
C0174	862.39
C0205	812.33

Preview: first 15 of 235 customers.

3.8 Contribution Proxy by Customer

SQL QUERY

```sql
SELECT
    customer_id,
    ROUND(
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN quantity * unit_selling_price
                     - discount_amount
                     - refund_amount
                     - payment_fee
                     - fulfilment_cost
                ELSE 0
            END
        ), 2
    ) AS contribution_proxy
FROM marketplace_transactions
GROUP BY customer_id
ORDER BY contribution_proxy DESC;
```

OUTPUT

customer_id	contribution_proxy
C0100	8,461.59
C0002	6,415.45
C0097	6,245.43
C0131	5,753.40
C0010	5,515.53
C0239	5,428.95
C0150	5,360.28
C0129	5,336.17
C0085	5,218.64
C0189	5,211.15
C0024	5,038.57
C0165	4,995.81
C0114	4,821.65
C0109	4,574.81
C0064	4,517.60

Preview: first 15 of 235 customers.

3.9 High-Value Transactions with Weak Post-Discount Economics

SQL QUERY

```sql
WITH transaction_metrics AS (
    SELECT
        order_id,
        customer_id,
        SUM(quantity * unit_selling_price - discount_amount)
            AS delivered_gmv,
        SUM(
            quantity * unit_selling_price
            - discount_amount
            - refund_amount
            - payment_fee
            - fulfilment_cost
        ) AS contribution_proxy
    FROM marketplace_transactions
    WHERE order_status = 'Delivered'
    GROUP BY order_id, customer_id
)
SELECT
    order_id,
    customer_id,
    ROUND(delivered_gmv, 2) AS delivered_gmv,
    ROUND(contribution_proxy, 2) AS contribution_proxy
FROM transaction_metrics
WHERE delivered_gmv >= (
    SELECT AVG(delivered_gmv)
    FROM transaction_metrics
)
AND contribution_proxy <= 0
ORDER BY delivered_gmv DESC;
```

OUTPUT

order_id	customer_id	delivered_gmv	contribution_proxy

Empty set: no delivered order combines above-average GMV with zero or negative contribution.

3.10 10% Delivered GMV Reduction Scenario Estimate

SQL QUERY

```sql
WITH category_metrics AS (
    SELECT
        product_category,
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN quantity * unit_selling_price
                     - discount_amount
                ELSE 0
            END
        ) AS delivered_gmv,
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN quantity * unit_selling_price
                     - discount_amount
                     - refund_amount
                     - payment_fee
                     - fulfilment_cost
                ELSE 0
            END
        ) AS contribution_proxy
    FROM marketplace_transactions
    GROUP BY product_category
)
SELECT
    product_category,
    ROUND(delivered_gmv, 2) AS delivered_gmv,
    ROUND(contribution_proxy, 2) AS contribution_proxy,
    ROUND(delivered_gmv * 0.10, 2) AS gmv_reduction_scenario,
    ROUND(contribution_proxy * 0.10, 2)
        AS contribution_proxy_at_risk
FROM category_metrics
ORDER BY contribution_proxy_at_risk DESC;
```

OUTPUT

product_category	delivered_gmv	contribution_proxy	gmv_reduction_scenario	contribution_proxy_at_risk
Electronics	108,362.46	95,966.49	10,836.25	9,596.65
Beauty	101,829.78	89,737.69	10,182.98	8,973.77
Home & Kitchen	91,523.91	79,640.35	9,152.39	7,964.04
Fashion	74,792.31	63,546.35	7,479.23	6,354.64
Sports	71,398.64	61,702.04	7,139.86	6,170.20

SECTION 4

Acquisition cohort quality

Business question. Build acquisition-month cohorts. Compare first-purchase contribution, 90-day cumulative contribution, and repeat-purchase rate by acquisition channel. Identify cohorts where first-purchase economics are positive but 90-day economics deteriorate, and flag incomplete 90-day windows.

4.0 Check Table Structure and Data Types

SQL QUERY

```
DESCRIBE marketplace_transactions;
```

Utility query. Returns the column names and data types of the table; it produces no analytical result table.

4.1 Check Date Format

SQL QUERY

```
SELECT customer_acquisition_date, order_timestamp
FROM marketplace_transactions
LIMIT 10;
```

OUTPUT

customer_ acquisition_ date	order_ timestamp
06-07-2025	14-08-2025
19-05-2025	12-02-2025
29-01-2023	30-09-2025
25-08-2025	20-09-2025
09-04-2024	07-04-2025
29-05-2023	20-12-2025
23-06-2024	17-12-2025
24-04-2024	10-03-2025
24-03-2024	28-03-2025
30-10-2024	19-04-2025

4.2 Build Acquisition-Month Cohorts

SQL QUERY

```sql
SELECT
    acquisition_channel,
    DATE_FORMAT(
        STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d'),
        '%Y-%m'
    ) AS cohort_month,
    COUNT(DISTINCT customer_id) AS acquired_customers
FROM marketplace_transactions
GROUP BY acquisition_channel, cohort_month
ORDER BY cohort_month, acquisition_channel;
```

OUTPUT

acquisition_ channel	cohort_month	acquired_ customers
Affiliate	2023-01	7
Organic	2023-01	9
Paid Search	2023-01	4
Referral	2023-01	4
Social	2023-01	8
Organic	2023-02	7
Paid Search	2023-02	3
Referral	2023-02	5
Social	2023-02	4
Affiliate	2023-03	3
Organic	2023-03	6
Paid Search	2023-03	4
Referral	2023-03	5
Social	2023-03	2
Organic	2023-04	6
Paid Search	2023-04	1
Referral	2023-04	5
Social	2023-04	8
Affiliate	2023-05	7
Organic	2023-05	4

Preview: first 20 of 172 cohort rows.

4.3 Identify First Delivered Purchase

SQL QUERY

```sql
WITH delivered_orders AS (
    SELECT
        customer_id,
        acquisition_channel,
        order_id,
        MIN(STR_TO_DATE(order_timestamp, '%Y-%m-%d %H:%i:%s')) AS order_date,
        SUM(
            quantity * unit_selling_price
            - discount_amount
            - refund_amount
            - payment_fee
            - fulfilment_cost
        ) AS contribution
    FROM marketplace_transactions
    WHERE order_status = 'Delivered'
    GROUP BY customer_id, acquisition_channel, order_id
),
ranked_orders AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date, order_id
        ) AS purchase_rank
    FROM delivered_orders
)
SELECT
    customer_id,
    acquisition_channel,
    order_id,
    order_date AS first_purchase_date,
    ROUND(contribution, 2) AS first_purchase_contribution
FROM ranked_orders
WHERE purchase_rank = 1
ORDER BY first_purchase_date;
```

OUTPUT

customer_id	acquisition_ channel	order_id	first_purchase_ date	first_purchase_ contribution
C0022	Affiliate	O00481	01-01-2025 00:00	93.02
C0048	Organic	O00136	01-01-2025 00:00	520.74
C0217	Paid Search	O00237	02-01-2025 00:00	166.57
C0012	Affiliate	O00445	03-01-2025 00:00	806.58
C0136	Paid Search	O00208	04-01-2025 00:00	368.97
C0139	Referral	O00051	04-01-2025 00:00	792.09
C0047	Paid Search	O00475	05-01-2025 00:00	774.50
C0152	Paid Search	O00583	05-01-2025 00:00	433.89
C0189	Paid Search	O00461	05-01-2025 00:00	1,064.83
C0064	Referral	O00143	06-01-2025 00:00	262.84
C0105	Social	O00587	06-01-2025 00:00	208.91
C0121	Organic	O00490	06-01-2025 00:00	475.88
C0221	Affiliate	O00201	06-01-2025 00:00	394.35
C0021	Organic	O00536	07-01-2025 00:00	1,615.95
C0183	Referral	O00309	07-01-2025 00:00	391.23
C0100	Referral	O00233	08-01-2025 00:00	391.75
C0058	Referral	O00501	11-01-2025 00:00	213.27
C0128	Organic	O00460	11-01-2025 00:00	428.53
C0151	Social	O00588	11-01-2025 00:00	1,015.01
C0150	Referral	O00469	12-01-2025 00:00	222.37

Preview: first 20 of 226 customers with a delivered first purchase.

4.4 Calculate 90-Day Cumulative Contribution

SQL QUERY

```sql
WITH delivered_orders AS (
    SELECT
        customer_id,
        acquisition_channel,
        order_id,
        MIN(STR_TO_DATE(order_timestamp, '%Y-%m-%d %H:%i:%s')) AS order_date,
        SUM(
            quantity * unit_selling_price
            - discount_amount
            - refund_amount
            - payment_fee
            - fulfilment_cost
        ) AS contribution
    FROM marketplace_transactions
    WHERE order_status = 'Delivered'
    GROUP BY customer_id, acquisition_channel, order_id
),
first_purchase AS (
    SELECT
        customer_id,
        MIN(order_date) AS first_purchase_date
    FROM delivered_orders
    GROUP BY customer_id
)
SELECT
    d.customer_id,
    ROUND(SUM(d.contribution), 2) AS contribution_90d
FROM delivered_orders d
JOIN first_purchase f
    ON d.customer_id = f.customer_id
WHERE d.order_date >= f.first_purchase_date
  AND d.order_date < DATE_ADD(f.first_purchase_date, INTERVAL 90 DAY)
GROUP BY d.customer_id
ORDER BY d.customer_id;
```

OUTPUT

customer_id	contribution_90d
C0001	311.98
C0002	4,366.68
C0003	676.27
C0004	2,659.27
C0005	1,065.38
C0006	482.68
C0007	461.03
C0009	108.01
C0010	708.08
C0011	677.10
C0012	806.58
C0013	1,077.39
C0014	238.08
C0016	917.52
C0017	680.24

Preview: first 15 of 226 customers.

4.5 Calculate Repeat-Purchase Rate by Acquisition Channel

SQL QUERY

```sql
SELECT
    acquisition_channel,
    ROUND(
        100.0 * SUM(
            CASE WHEN delivered_orders >= 2 THEN 1 ELSE 0 END
        ) / NULLIF(COUNT(*), 0),
        2
    ) AS repeat_purchase_rate_pct
FROM (
    SELECT
        customer_id,
        acquisition_channel,
        COUNT(DISTINCT order_id) AS delivered_orders
    FROM marketplace_transactions
    WHERE order_status = 'Delivered'
    GROUP BY customer_id, acquisition_channel
) AS c
GROUP BY acquisition_channel
ORDER BY acquisition_channel;
```

OUTPUT

acquisition_channel	repeat_purchase_rate_pct
Affiliate	18.69
Organic	31.62
Paid Search	26.00
Referral	39.22
Social	24.55

4.6 Final Acquisition Cohort Quality

SQL QUERY

```
WITH delivered_orders AS (
    SELECT
        customer_id,
        acquisition_channel,
        order_id,
        MIN(STR_TO_DATE(order_timestamp, '%Y-%m-%d %H:%i:%s')) AS order_date,
        SUM(
            quantity * unit_selling_price
            - discount_amount
            - refund_amount
            - payment_fee
            - fulfilment_cost
        ) AS contribution
    FROM marketplace_transactions
    WHERE order_status = 'Delivered'
    GROUP BY customer_id, acquisition_channel, order_id
),

ranked_orders AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date, order_id
        ) AS purchase_rank
    FROM delivered_orders
),

first_purchase AS (
    SELECT
        customer_id,
        acquisition_channel,
        order_date AS first_purchase_date,
        contribution AS first_purchase_contribution
    FROM ranked_orders
    WHERE purchase_rank = 1
),

customer_cohorts AS (
    SELECT
        customer_id,
        acquisition_channel,
        DATE_FORMAT(
            STR_TO_DATE(
                MIN(customer_acquisition_date),
                '%Y-%m-%d'
            ),
            '%Y-%m'
        ) AS cohort_month
    FROM marketplace_transactions
    GROUP BY customer_id, acquisition_channel
),
```

```sql
customer_metrics AS (
    SELECT
        f.customer_id,
        f.acquisition_channel,
        c.cohort_month,
        f.first_purchase_date,
        f.first_purchase_contribution,
        SUM(
            CASE
                WHEN d.order_date >= f.first_purchase_date
                 AND d.order_date < DATE_ADD(
                     f.first_purchase_date, INTERVAL 90 DAY
                 )
                THEN d.contribution
                ELSE 0
            END
        ) AS contribution_90d,
        COUNT(DISTINCT d.order_id) AS delivered_orders,
        MAX(d.order_date) AS last_observed_order_date
    FROM first_purchase f
    JOIN customer_cohorts c
        ON f.customer_id = c.customer_id
       AND f.acquisition_channel = c.acquisition_channel
    JOIN delivered_orders d
        ON f.customer_id = d.customer_id
       AND f.acquisition_channel = d.acquisition_channel
    GROUP BY
        f.customer_id,
        f.acquisition_channel,
        c.cohort_month,
        f.first_purchase_date,
        f.first_purchase_contribution
),

data_cutoff AS (
    SELECT MAX(order_date) AS max_order_date
    FROM delivered_orders
)

SELECT
    cm.acquisition_channel,
    cm.cohort_month,
    COUNT(DISTINCT cm.customer_id) AS acquired_customers,
    ROUND(SUM(cm.first_purchase_contribution), 2)
        AS first_purchase_contribution
```

```sql
    ROUND(
        CASE
            WHEN DATE_ADD(
                STR_TO_DATE(CONCAT(cm.cohort_month, '-01'), '%Y-%m-%d'),
                INTERVAL 3 MONTH
            ) <= dc.max_order_date
            THEN SUM(cm.contribution_90d)
            ELSE NULL
        END,
        2
    ) AS contribution_90d,

    CASE
        WHEN DATE_ADD(
            STR_TO_DATE(CONCAT(cm.cohort_month, '-01'), '%Y-%m-%d'),
            INTERVAL 3 MONTH
        ) <= dc.max_order_date
        THEN ROUND(
            100.0 * SUM(
                CASE WHEN cm.delivered_orders >= 2 THEN 1 ELSE 0 END
            ) / NULLIF(COUNT(DISTINCT cm.customer_id), 0),
            2
        )
        ELSE NULL
    END AS repeat_purchase_rate_pct,

    CASE
        WHEN DATE_ADD(
```

```
            STR_TO_DATE(CONCAT(cm.cohort_month, '-01'), '%Y-%m-%d'),
            INTERVAL 3 MONTH
        ) <= dc.max_order_date
        THEN 'Yes'
        ELSE 'No'
    END AS cohort_complete

FROM customer_metrics cm
CROSS JOIN data_cutoff dc
GROUP BY
    cm.acquisition_channel,
    cm.cohort_month,
    dc.max_order_date
ORDER BY
    cm.cohort_month,
    cm.acquisition_channel;
```

OUTPUT

acquisition_channel	cohort_month	acquired_customers	first_purchase_contribution	contribution_90d	repeat_purchase_rate_pct	cohort_complete
Affiliate	2023-01	4	1,693.52	2,329.14	75.00	Yes
Organic	2023-01	1	244.72	244.72	0.00	Yes
Paid Search	2023-01	2	446.19	3,787.50	50.00	Yes
Referral	2023-01	2	1,454.99	2,003.35	100.00	Yes
Social	2023-01	4	7,666.47	8,205.68	75.00	Yes
Organic	2023-02	2	1,823.72	2,298.58	100.00	Yes
Paid Search	2023-02	1	104.04	104.04	0.00	Yes
Referral	2023-02	2	741.08	929.25	50.00	Yes
Affiliate	2023-03	1	146.52	146.52	0.00	Yes
Organic	2023-03	2	64.68	64.68	0.00	Yes
Paid Search	2023-03	3	4,823.46	4,823.46	33.33	Yes
Referral	2023-03	3	612.86	612.86	66.67	Yes
Organic	2023-04	3	2,499.38	2,492.27	100.00	Yes
Referral	2023-04	3	2,488.66	2,550.77	66.67	Yes
Social	2023-04	3	1,894.94	1,894.94	33.33	Yes
Affiliate	2023-05	3	858.75	4,076.53	66.67	Yes
Organic	2023-05	2	703.58	703.58	0.00	Yes
Referral	2023-05	2	1,177.94	2,066.23	100.00	Yes
Social	2023-05	2	1,308.49	1,689.78	100.00	Yes
Affiliate	2023-06	1	503.15	1,352.62	100.00	Yes
Organic	2023-06	1	86.38	86.38	0.00	Yes
Paid Search	2023-06	4	3,614.40	3,614.40	50.00	Yes
Referral	2023-06	2	1,893.70	2,689.16	100.00	Yes
Social	2023-06	1	2,081.88	2,081.88	0.00	Yes
Affiliate	2023-07	3	311.71	311.71	0.00	Yes
Organic	2023-07	4	1,299.77	1,525.60	25.00	Yes

acquisition_channel	cohort_month	acquired_customers	first_purchase_contribution	contribution_90d	repeat_purchase_rate_pct	cohort_complete
Paid Search	2023-07	1	320.62	320.62	0.00	Yes
Social	2023-07	1	687.24	687.24	0.00	Yes
Paid Search	2023-08	2	115.43	115.43	0.00	Yes
Referral	2023-08	3	821.67	821.67	66.67	Yes
Social	2023-08	3	2,033.65	2,033.65	0.00	Yes
Affiliate	2023-09	1	605.56	605.56	0.00	Yes
Organic	2023-09	4	3,804.77	3,828.52	25.00	Yes
Referral	2023-09	1	1,186.00	1,186.00	0.00	Yes
Social	2023-09	3	1,143.97	1,631.06	33.33	Yes
Affiliate	2023-10	4	1,366.38	1,959.71	25.00	Yes
Organic	2023-10	3	1,238.11	1,353.67	66.67	Yes
Paid Search	2023-10	3	579.38	977.48	33.33	Yes
Referral	2023-10	3	411.09	735.87	100.00	Yes
Social	2023-10	2	349.78	786.29	50.00	Yes
Affiliate	2023-11	3	1,658.79	1,658.79	0.00	Yes
Paid Search	2023-11	2	2,345.09	2,345.09	0.00	Yes
Referral	2023-11	1	495.22	495.22	100.00	Yes
Affiliate	2023-12	1	354.25	354.25	100.00	Yes
Organic	2023-12	3	2,036.16	6,643.19	66.67	Yes
Referral	2023-12	1	522.12	522.12	0.00	Yes
Social	2023-12	2	5,402.99	5,402.99	50.00	Yes
Affiliate	2024-01	4	1,341.12	1,787.14	50.00	Yes
Organic	2024-01	1	249.76	249.76	0.00	Yes
Paid Search	2024-01	1	774.50	774.50	0.00	Yes
Referral	2024-01	2	782.31	1,335.54	50.00	Yes
Social	2024-01	3	2,170.75	2,170.75	0.00	Yes
Organic	2024-02	1	248.66	248.66	100.00	Yes
Paid Search	2024-02	2	850.71	850.71	50.00	Yes
Referral	2024-02	2	307.08	461.87	50.00	Yes
Affiliate	2024-03	2	1,251.87	1,251.87	0.00	Yes
Organic	2024-03	5	3,572.67	3,764.09	60.00	Yes
Social	2024-03	1	134.96	134.96	100.00	Yes
Organic	2024-04	2	388.59	970.78	50.00	Yes
Paid Search	2024-04	1	1,064.83	1,064.83	0.00	Yes
Social	2024-04	1	208.91	649.16	100.00	Yes
Affiliate	2024-05	1	32.88	32.88	0.00	Yes

acquisition_channel	cohort_month	acquired_customers	first_purchase_contribution	contribution_90d	repeat_purchase_rate_pct	cohort_complete
Organic	2024-05	1	1,868.75	1,868.75	0.00	Yes
Organic	2024-06	2	1,624.35	1,624.35	50.00	Yes
Paid Search	2024-06	1	2,021.40	2,021.40	0.00	Yes
Social	2024-06	1	115.90	115.90	0.00	Yes
Affiliate	2024-07	2	972.03	972.03	0.00	Yes
Paid Search	2024-07	1	160.13	160.13	0.00	Yes
Social	2024-07	1	912.19	912.19	0.00	Yes
Referral	2024-08	2	471.66	1,125.81	50.00	Yes
Affiliate	2024-09	3	1,643.40	1,643.40	33.33	Yes
Paid Search	2024-09	5	2,387.52	2,387.52	20.00	Yes
Affiliate	2024-10	2	1,045.25	1,045.25	0.00	Yes
Paid Search	2024-10	2	807.13	807.13	0.00	Yes
Referral	2024-10	2	2,072.57	2,901.85	100.00	Yes
Affiliate	2024-11	3	1,278.24	1,278.24	33.33	Yes
Organic	2024-11	1	136.66	136.66	0.00	Yes
Referral	2024-11	1	84.95	84.95	0.00	Yes
Social	2024-11	1	0.66	0.66	0.00	Yes
Organic	2024-12	3	603.70	603.70	0.00	Yes
Paid Search	2024-12	1	681.87	681.87	100.00	Yes
Social	2024-12	1	747.33	747.33	0.00	Yes
Affiliate	2025-01	1	1,026.35	1,026.35	0.00	Yes
Organic	2025-01	1	475.88	475.88	0.00	Yes
Referral	2025-01	1	116.32	116.32	0.00	Yes
Organic	2025-02	2	2,013.37	2,013.37	0.00	Yes
Referral	2025-02	2	762.87	762.87	0.00	Yes
Social	2025-02	2	423.00	423.00	0.00	Yes
Affiliate	2025-03	1	113.27	113.27	0.00	Yes
Paid Search	2025-03	1	284.33	284.33	0.00	Yes
Referral	2025-03	2	204.24	204.24	0.00	Yes
Affiliate	2025-04	2	820.68	820.68	0.00	Yes
Organic	2025-04	2	971.29	971.29	0.00	Yes
Paid Search	2025-04	6	3,939.31	3,939.31	16.67	Yes
Referral	2025-04	1	156.73	766.79	100.00	Yes
Social	2025-04	1	134.29	134.29	100.00	Yes
Organic	2025-05	1	310.91	310.91	0.00	Yes
Paid Search	2025-05	3	927.94	1,300.73	66.67	Yes

acquisition_channel	cohort_month	acquired_customers	first_purchase_contribution	contribution_90d	repeat_purchase_rate_pct	cohort_complete
Social	2025-05	1	205.65	205.65	0.00	**Yes**
Paid Search	2025-06	1	1,320.70	1,320.70	0.00	**Yes**
Referral	2025-06	1	1,189.28	1,189.28	100.00	**Yes**
Social	2025-06	1	587.35	587.35	0.00	**Yes**
Affiliate	2025-07	1	100.41	100.41	0.00	**Yes**
Referral	2025-07	1	278.36	278.36	0.00	**Yes**
Social	2025-07	1	97.00	97.00	100.00	**Yes**
Affiliate	2025-08	1	630.21	630.21	0.00	**Yes**
Referral	2025-08	1	223.96	223.96	0.00	**Yes**
Social	2025-08	1	42.14	42.14	0.00	**Yes**
Affiliate	2025-09	1	324.59	324.59	0.00	**Yes**
Paid Search	2025-09	1	472.57	472.57	0.00	**Yes**
Referral	2025-09	1	391.23	391.23	0.00	**Yes**
Social	2025-09	2	1,770.77	1,770.77	0.00	**Yes**
Affiliate	2025-10	1	59.75	*NULL*	*NULL*	No
Organic	2025-10	2	1,665.13	*NULL*	*NULL*	No
Paid Search	2025-10	1	102.76	*NULL*	*NULL*	No
Referral	2025-10	1	777.48	*NULL*	*NULL*	No
Social	2025-10	1	429.30	*NULL*	*NULL*	No
Organic	2025-11	1	199.39	*NULL*	*NULL*	No
Social	2025-11	2	368.97	*NULL*	*NULL*	No

NULL in contribution_90d and repeat_purchase_rate_pct marks an incomplete 90-day window (cohort_complete = No).

SECTION 5

Commercial downside scenario

Business question. Quantify current monthly contribution by category. Calculate the break-even delivered-GMV decline that eliminates the benefit of a 10% reduction in discount expense. Identify categories with the smallest tolerance for volume decline.

5.1 Current Monthly Contribution by Category

SQL QUERY

```sql
SELECT
    DATE_FORMAT(
        STR_TO_DATE(order_timestamp, '%Y-%m-%d %H:%i:%s'),
        '%Y-%m'
    ) AS order_month,
    product_category,
    ROUND(
        SUM(
            CASE
                WHEN order_status = 'Delivered' THEN
                    quantity * unit_selling_price
                    - discount_amount
                    - refund_amount
                    - payment_fee
                    - fulfilment_cost
                ELSE 0
            END
        ), 2
    ) AS current_contribution
FROM marketplace_transactions
GROUP BY order_month, product_category
ORDER BY order_month, product_category;
```

OUTPUT

order_month	product_ category	current_ contribution
2025-01	Beauty	4,672.73
2025-01	Electronics	9,336.66
2025-01	Fashion	9,333.20
2025-01	Home & Kitchen	7,755.91
2025-01	Sports	5,368.54
2025-02	Beauty	7,477.25
2025-02	Electronics	8,716.41
2025-02	Fashion	3,702.05
2025-02	Home & Kitchen	8,289.20
2025-02	Sports	9,418.89
2025-03	Beauty	8,686.32
2025-03	Electronics	2,738.47
2025-03	Fashion	6,372.60
2025-03	Home & Kitchen	17,557.59
2025-03	Sports	8,066.84

Preview: first 15 of 60 month and category rows.

5.2 Calculate 10% Discount Saving

SQL QUERY

```sql
SELECT
    DATE_FORMAT(
        STR_TO_DATE(order_timestamp, '%Y-%m-%d %H:%i:%s'),
        '%Y-%m'
    ) AS order_month,
    product_category,
    ROUND(
        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN discount_amount * 0.10
                ELSE 0
            END
        ), 2
    ) AS discount_saving_scenario
FROM marketplace_transactions
GROUP BY order_month, product_category
ORDER BY order_month, product_category;
```

OUTPUT

order_month	product_ category	discount_ saving_ scenario
2025-01	Beauty	90.10
2025-01	Electronics	160.01
2025-01	Fashion	138.35
2025-01	Home & Kitchen	196.15
2025-01	Sports	113.45
2025-02	Beauty	313.80
2025-02	Electronics	180.53
2025-02	Fashion	105.65
2025-02	Home & Kitchen	183.42
2025-02	Sports	162.14
2025-03	Beauty	326.65
2025-03	Electronics	87.19
2025-03	Fashion	192.37
2025-03	Home & Kitchen	395.13
2025-03	Sports	226.58

Preview: first 15 of 60 month and category rows.

5.3 Break-even GMV Decline

SQL QUERY

```sql
WITH monthly_category AS (
    SELECT
        DATE_FORMAT(
            STR_TO_DATE(order_timestamp, '%Y-%m-%d %H:%i:%s'),
            '%Y-%m'
        ) AS order_month,
        product_category,

        SUM(
            CASE
                WHEN order_status = 'Delivered' THEN
                    quantity * unit_selling_price
                    - discount_amount
                    - refund_amount
                    - payment_fee
                    - fulfilment_cost
                ELSE 0
            END
        ) AS current_contribution,

        SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN discount_amount * 0.10
                ELSE 0
            END
        ) AS discount_saving

    FROM marketplace_transactions
    GROUP BY order_month, product_category
)

SELECT
    order_month,
    product_category,
    ROUND(current_contribution, 2) AS current_contribution,
    ROUND(discount_saving, 2) AS discount_saving_scenario,
    ROUND(current_contribution, 2)
        AS contribution_at_risk_per_100pct_gmv_decline,

    ROUND(
        100.0 * discount_saving
        / NULLIF(current_contribution, 0),
        2
    ) AS break_even_gmv_decline_pct

FROM monthly_category
ORDER BY break_even_gmv_decline_pct ASC;
```

OUTPUT

order_month	product_ category	current_ contribution	discount_ saving_ scenario	contribution_ at_risk_per_ 100pct_gmv_ decline	break_even_gmv_ decline_pct
2025-09	Sports	4,493.13	46.36	4,493.13	1.03
2025-04	Sports	3,510.46	38.93	3,510.46	1.11
2025-06	Electronics	2,210.92	26.89	2,210.92	1.22
2025-07	Electronics	8,502.40	113.00	8,502.40	1.33
2025-06	Home & Kitchen	5,512.99	74.18	5,512.99	1.35
2025-10	Electronics	7,355.89	105.41	7,355.89	1.43
2025-08	Home & Kitchen	3,651.67	52.69	3,651.67	1.44
2025-01	Fashion	9,333.20	138.35	9,333.20	1.48
2025-08	Electronics	6,615.32	101.41	6,615.32	1.53
2025-05	Beauty	14,015.27	221.37	14,015.27	1.58
2025-08	Fashion	4,997.29	82.11	4,997.29	1.64
2025-11	Sports	4,193.21	70.46	4,193.21	1.68
2025-01	Electronics	9,336.66	160.01	9,336.66	1.71
2025-06	Sports	7,119.00	122.10	7,119.00	1.72
2025-02	Sports	9,418.89	162.14	9,418.89	1.72

Preview: the 15 tightest of 60 month and category rows (lowest break-even first).

5.4 Category Break-Even Summary

SQL QUERY

```sql
WITH category_scenario AS (
    SELECT
        product_category,

        SUM(
            CASE
                WHEN order_status = 'Delivered' THEN
                    quantity * unit_selling_price
                    - discount_amount
                    - refund_amount
                    - payment_fee
                    - fulfilment_cost
                ELSE 0
            END
        ) AS current_contribution,

        0.10 * SUM(
            CASE
                WHEN order_status = 'Delivered'
                THEN discount_amount
                ELSE 0
            END
        ) AS discount_saving,

        SUM(
            CASE
                WHEN order_status = 'Delivered' THEN
                    quantity * unit_selling_price
                    - discount_amount
                    - refund_amount
                    - payment_fee
                    - fulfilment_cost
                ELSE 0
            END
        ) AS contribution_at_risk

    FROM marketplace_transactions
    GROUP BY product_category
)

SELECT
    product_category,
    ROUND(current_contribution, 2) AS current_contribution,
    ROUND(discount_saving, 2) AS discount_saving_scenario,
    ROUND(contribution_at_risk, 2)
        AS contribution_at_risk_per_100pct_gmv_decline,

    ROUND(
        100.0 * discount_saving
        / NULLIF(contribution_at_risk, 0),
        2
    ) AS break_even_gmv_decline_pct

FROM category_scenario
ORDER BY break_even_gmv_decline_pct ASC;
```

OUTPUT

product_ category	current_ contribution	discount_ saving_ scenario	contribution_ at_risk_per_ 100pct_gmv_ decline	break_even_gmv_ decline_pct
Sports	61,702.04	1,288.46	61,702.04	2.09
Electronics	95,966.49	2,139.85	95,966.49	2.23
Fashion	63,546.35	1,796.34	63,546.35	2.83
Home & Kitchen	79,640.35	2,264.34	79,640.35	2.84
Beauty	89,737.69	2,705.58	89,737.69	3.01

End of case study